

Deep Learning Notes

Module 1: Introduction to Deep Learning

1.1 Fundamentals of Deep Learning

1.1.1 What is Deep Learning?

Definition

Deep Learning (DL) is a subset of **Machine Learning (ML)** that uses **Artificial Neural Networks (ANNs)** with multiple hidden layers to automatically learn patterns and features from large amounts of data.

Unlike traditional Machine Learning, Deep Learning does **not require manual feature extraction**. It automatically learns important features directly from raw data.

Simple Definition:

Deep Learning is a technique that enables computers to learn complex patterns from data using multi-layered neural networks.

Key Points

- Inspired by the structure and working of the **human brain**.
 - Uses **Artificial Neural Networks (ANNs)** with many hidden layers.
 - Learns features automatically.
 - Performs well on large datasets.
 - Improves performance as more data becomes available.
 - Requires high computational power (GPU/TPU).
 - Widely used in AI applications like image recognition, speech recognition, and language translation.
-

Real-Life Example

Imagine teaching a child to recognize a **cat**.

Traditional Machine Learning:

- You manually tell the computer:
 - Cats have ears.
 - Cats have whiskers.
 - Cats have four legs.

Deep Learning:

- You simply provide **thousands of cat images**.
 - The model automatically learns:
 - Eyes
 - Fur texture
 - Ear shape
 - Body structure
 - Tail
 - Finally, it identifies whether a new image contains a cat.
-

Basic Working

Input Data



Neural Network
(Many Hidden Layers)



Learns Features
Automatically



Prediction

Characteristics

- Learns automatically from data
 - Handles structured and unstructured data
 - Requires large datasets
 - Requires powerful hardware
 - High prediction accuracy
 - Reduces manual feature engineering
-

1.1.2 Artificial Intelligence vs Machine Learning vs Deep Learning

Relationship

Artificial Intelligence (AI)



Machine Learning (ML)



Deep Learning (DL)

Deep Learning is a subset of Machine Learning, and Machine Learning is a subset of Artificial Intelligence.

Artificial Intelligence (AI)

Definition

Artificial Intelligence is the field of creating machines that can perform tasks requiring human intelligence.

Examples

- Chatbots
- Self-driving cars
- Smart assistants
- Robotics

Goal

Make machines think, reason, and make decisions like humans.

Machine Learning (ML)

Definition

Machine Learning is a branch of AI where computers learn patterns from data to make predictions without being explicitly programmed.

Examples

- Spam email detection
- Movie recommendation systems

- House price prediction

Feature Extraction

Manual (performed by humans).

Deep Learning (DL)

Definition

Deep Learning is a branch of Machine Learning that uses deep neural networks to automatically learn features from data.

Examples

- Face recognition
- Speech recognition
- Medical image analysis
- ChatGPT
- Image generation

Feature Extraction

Automatic.

Comparison Table

Feature	AI	ML	DL
Full Form	Artificial Intelligence	Machine Learning	Deep Learning
Subset Of	—	AI	ML
Learns from Data	Sometimes	Yes	Yes
Feature Extraction	Rule-based/Manual	Manual	Automatic
Data Requirement	Low to Medium	Medium	Large
Hardware Requirement	Low	Medium	High
Accuracy	Moderate	High	Very High
Best For	General intelligent tasks	Prediction problems	Complex tasks (images, speech, text)

1.1.3 Why Deep Learning?

Deep Learning is used because it can solve complex problems that traditional Machine Learning struggles with.

Reasons

- Automatically learns features from raw data.
 - Handles very large datasets efficiently.
 - Achieves high accuracy for complex tasks.
 - Works well with images, audio, video, and text.
 - Improves as more data becomes available.
 - Reduces the need for manual feature engineering.
-

Example

Traditional ML:

- Human extracts image features.

Deep Learning:

- Model learns features automatically.
-

1.1.4 History and Evolution

Year	Development
1943	First Artificial Neuron (McCulloch & Pitts)
1958	Perceptron introduced by Frank Rosenblatt
1986	Backpropagation algorithm popularized
1998	Convolutional Neural Networks (LeNet)
2006	Deep Learning revived (Geoffrey Hinton)
2012	AlexNet won ImageNet competition
2016	AlphaGo defeated world champion
Present	LLMs, Generative AI, Autonomous Systems

1.1.5 Applications of Deep Learning

Deep Learning is widely used in many industries.

Computer Vision

- Face recognition
- Object detection
- Image classification
- Medical image analysis

Natural Language Processing (NLP)

- Chatbots
- Machine translation
- Text summarization
- Sentiment analysis

Speech Processing

- Voice assistants
- Speech-to-text
- Speaker identification

Healthcare

- Disease diagnosis
- Drug discovery
- Cancer detection

Finance

- Fraud detection
- Credit scoring
- Stock prediction

Autonomous Vehicles

- Lane detection
- Traffic sign recognition
- Obstacle detection

Entertainment

- Movie recommendations
- Music recommendations
- Content personalization

1.1.6 Advantages of Deep Learning

- Automatic feature extraction.
 - High accuracy on complex tasks.
 - Handles large and diverse datasets.
 - Learns complex non-linear relationships.
 - Scales well with more data.
 - Supports end-to-end learning.
 - Powers modern AI applications.
-

1.1.7 Limitations of Deep Learning

- Requires a large amount of labeled data.
 - Needs expensive hardware (GPU/TPU).
 - Training can be time-consuming.
 - Models are often difficult to interpret ("black box").
 - Hyperparameter tuning is complex.
 - High energy and computational cost.
-

1.1.8 Challenges of Deep Learning

Data Availability

Obtaining sufficient high-quality labeled data can be difficult.

Overfitting

The model memorizes training data instead of learning general patterns.

Computational Cost

Training deep networks requires powerful hardware and significant time.

Explainability

It is often hard to understand why a deep model made a particular prediction.

Bias

Models can learn and amplify biases present in the training data.

Hyperparameter Selection

Choosing learning rate, batch size, epochs, and network architecture requires experimentation.

1.1.9 Future Scope of Deep Learning

Deep Learning continues to evolve rapidly and is expected to play a major role in future technologies.

Emerging Areas

- Generative AI (text, images, videos)
- Large Language Models (LLMs)
- Autonomous robots
- Smart healthcare systems
- Drug discovery
- Personalized education
- Cybersecurity
- Climate prediction
- Smart manufacturing
- Edge AI (running models on mobile and IoT devices)

1.2 Deep Learning Workflow

Definition

The **Deep Learning Workflow** is the step-by-step process of building, training, deploying, and maintaining a deep learning model.

Problem Definition



Data Collection



Data Preprocessing



Dataset Splitting



Model Selection



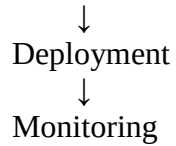
Training



Validation



Testing



1. Problem Definition

Definition

Clearly identify the problem that needs to be solved.

Purpose

- Understand the objective.
- Decide the type of task (Classification, Regression, etc.).
- Define success criteria.

Example

- Spam Email Detection
 - Face Recognition
 - House Price Prediction
-

2. Data Collection

Definition

Collect relevant and high-quality data for training the model.

Data Sources

- Kaggle
- Company Database
- APIs
- Sensors
- Web Scraping

Good Dataset

- Large
- Clean
- Balanced
- Relevant

3. Data Preprocessing

Definition

Preparing raw data before training.

Common Steps

- Remove missing values
- Remove duplicates
- Normalize/Standardize data
- Encode categorical values
- Resize images
- Tokenize text

Benefits

- Improves accuracy
 - Reduces noise
 - Faster training
-

4. Dataset Splitting

Definition

Divide data into different sets.

Dataset	Purpose	Size
Training	Learn patterns	70–80%
Validation	Tune model	10–15%
Testing	Final evaluation	10–15%

Why?

- Prevents overfitting
 - Evaluates model fairly
-

5. Model Selection

Definition

Choose the appropriate deep learning model based on the problem.

Examples

Problem	Model
Image	CNN
Text	RNN / Transformer
Speech	LSTM
Image Generation	GAN

6. Training

Definition

The model learns patterns by updating its weights using training data.

Training Process

1. Input data
2. Prediction
3. Calculate Loss
4. Backpropagation
5. Update Weights

Important Terms

- **Epoch:** One complete pass through the dataset.
 - **Batch:** Small group of samples processed together.
 - **Iteration:** One weight update.
-

7. Validation

Definition

Evaluate the model during training using validation data.

Purpose

- Tune hyperparameters
- Detect overfitting

- Select the best model
-

8. Testing

Definition

Evaluate the final model using unseen test data.

Common Metrics

- Accuracy
 - Precision
 - Recall
 - F1-Score
-

9. Deployment

Definition

Make the trained model available for real-world use.

Examples

- Mobile App
 - Website
 - Cloud API
 - IoT Device
-

10. Monitoring

Definition

Continuously track the model after deployment.

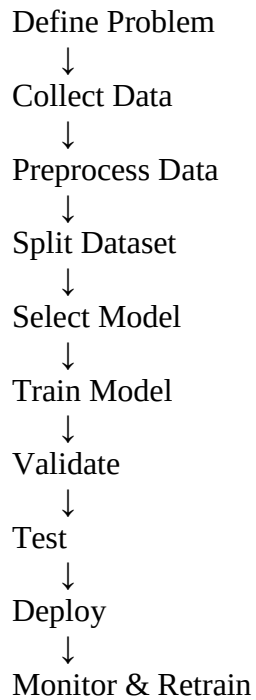
Monitor

- Accuracy
- Errors
- Data Drift
- Response Time

Why?

Models may lose accuracy as new data changes over time.

Workflow Summary



1.3 Hardware for Deep Learning

Definition

Deep learning requires powerful hardware because training neural networks involves billions of mathematical calculations.

1. CPU (Central Processing Unit)

Definition

General-purpose processor that executes program instructions.

Features

- Few powerful cores
- Sequential processing
- Suitable for small models

Advantages

- Low cost
- Easy to use
- Good for preprocessing

Limitations

- Slow for deep learning training
-

2. GPU (Graphics Processing Unit)

Definition

Processor designed for parallel computing, making it ideal for deep learning.

Features

- Thousands of cores
- Fast matrix calculations
- Supports parallel processing

Advantages

- Faster than CPU
- Ideal for training large models

Limitation

- More expensive
 - Higher power consumption
-

3. TPU (Tensor Processing Unit)

Definition

Google's custom hardware built specifically for AI and deep learning.

Features

- Optimized for tensor operations
- Very fast
- Mostly used in Google Cloud

Advantages

- High speed
 - Energy efficient
-

4. CUDA

Definition

CUDA is NVIDIA's platform that enables GPUs to perform general-purpose computations, including deep learning.

Benefits

- Accelerates training
- Supports TensorFlow and PyTorch
- Uses GPU efficiently

Note: CUDA works only with NVIDIA GPUs.

5. Parallel Computing

Definition

Performing multiple calculations simultaneously using multiple processors.

Benefits

- Faster training
 - Handles large datasets
 - Supports complex models
-

6. Memory Requirements

Types

- **RAM:** Stores datasets and program data.
- **VRAM:** GPU memory used during model training.

Memory Depends On

- Batch size
- Model size
- Input size
- Number of layers

Tip

Larger models require more VRAM.

Module 2: Neural Network Fundamentals

Neural Networks are the **foundation of Deep Learning**. They are inspired by the **human brain** and consist of interconnected neurons that learn patterns from data.

2.1 Biological Neuron

Human Brain

The **human brain** contains about **86 billion neurons** connected through synapses. These neurons communicate by transmitting electrical and chemical signals, enabling learning, memory, and decision-making.

Functions

- Process information
 - Learn from experience
 - Store memories
 - Make decisions
-

Biological Neuron

A biological neuron is the **basic functional unit of the brain**.

Parts of a Neuron

- **Dendrites:** Receive signals from other neurons.
- **Cell Body (Soma):** Processes the received signals.
- **Axon:** Carries the processed signal.
- **Axon Terminal:** Sends the signal to the next neuron through synapses.

Dendrites → Cell Body → Axon → Next Neuron

Signal Transmission

The communication between neurons follows these steps:

1. Dendrites receive signals.
 2. Cell body processes the signals.
 3. If the signal is strong enough, it generates an impulse.
 4. The impulse travels through the axon.
 5. The signal is passed to the next neuron.
-

2.2 Artificial Neuron

Definition

An **Artificial Neuron** is the mathematical model of a biological neuron. It receives inputs, performs calculations, and produces an output.

Inputs → Weights → Weighted Sum + Bias → Activation Function → Output

Components of an Artificial Neuron

1. Inputs (x)

Input features given to the neuron.

Example:

- Age
 - Salary
 - Height
-

2. Weights (w)

Weights determine the **importance** of each input.

- Higher weight → More influence
 - Lower weight → Less influence
-

3. Bias (b)

Bias shifts the output and helps the model fit data more accurately.

Think of bias as an **extra adjustable value** added to the weighted sum.

4. Weighted Sum

The neuron multiplies each input by its corresponding weight and adds them together.

Formula

$$z = \sum(w_i x_i) + b$$

Where:

- x_i = Input
 - w_i = Weight
 - b = Bias
 - z = Weighted sum
-

5. Activation Function

Applies a function to the weighted sum to decide the neuron's output.

Purpose:

- Introduces non-linearity.
- Enables learning of complex patterns.

Examples:

- Sigmoid
- ReLU
- Tanh

- Softmax
-

6. Output

The final value produced by the neuron and passed to the next layer.

2.3 Perceptron

Definition

A **Perceptron** is the **simplest neural network**, introduced by **Frank Rosenblatt (1958)**. It consists of a single neuron used mainly for binary classification.

Single Layer Perceptron

Contains:

- Input Layer
- Output Layer

No hidden layer.

Inputs → Perceptron → Output

Used for:

- Yes/No
 - True/False
 - Spam/Not Spam
-

Learning Algorithm

The perceptron learns by adjusting its weights based on prediction errors.

Steps

1. Initialize weights.
2. Calculate output.

3. Compare with actual output.
4. Update weights if prediction is incorrect.
5. Repeat until error is minimized.

Decision Boundary

A **decision boundary** separates different classes.

Example:

Class A	Class B
Decision Boundary

A perceptron can only create a **linear decision boundary** (straight line or plane).

Perceptron Equation

$$y = f(\sum(w_i x_i) + b)$$

Where:

- x = Inputs
 - w = Weights
 - b = Bias
 - f = Activation function
 - y = Output
-

XOR Problem

The **XOR (Exclusive OR)** problem cannot be solved by a single-layer perceptron because it is **not linearly separable**.

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Solution:

- Use a **Multi-Layer Perceptron (MLP)** with hidden layers.
-

Limitations of Perceptron

- Solves only **linear problems**.
 - Cannot solve XOR.
 - Limited learning capability.
 - Simple architecture.
-

2.4 Multi-Layer Perceptron (MLP)

Definition

A **Multi-Layer Perceptron (MLP)** is a neural network with **one or more hidden layers**, enabling it to solve complex, non-linear problems.

Input Layer



Hidden Layer(s)



Output Layer

1. Input Layer

- Receives input features.
 - Passes data to the hidden layer.
 - Does not perform computations.
-

2. Hidden Layer

- Performs computations using weights and activation functions.
 - Learns important patterns and relationships in the data.
 - An MLP can have multiple hidden layers.
-

3. Output Layer

- Produces the final prediction.
 - Number of neurons depends on the problem:
 - 1 neuron → Binary classification.
 - Multiple neurons → Multi-class classification.
-

Fully Connected Networks

In a **Fully Connected (Dense) Network**, every neuron in one layer is connected to **every neuron** in the next layer.

Advantages

- Learns complex relationships.
- High flexibility.

Disadvantage

- Large number of parameters.
-

Feed Forward Networks

A **Feed Forward Neural Network (FFNN)** passes information in **one direction only**:

Input → Hidden → Output

There are **no loops or feedback connections**.

Universal Approximation Theorem

This theorem states that:

A neural network with **at least one hidden layer** and enough neurons can approximate almost any continuous mathematical function.

Importance

- Shows why neural networks are powerful.
 - Explains their ability to solve complex real-world problems.
-

2.5 Neural Network Architecture

Neural network architecture refers to how the network is designed, including the number of layers, neurons, and connections.

1. Number of Layers

Types:

- **Input Layer**
- **Hidden Layer(s)**
- **Output Layer**

More hidden layers allow the model to learn more complex features but also increase computational cost.

2. Number of Neurons

Neurons are the processing units in each layer.

- More neurons → Higher learning capacity.
 - Too many neurons → Risk of overfitting.
-

3. Width vs Depth

Width	Depth
Number of neurons in a layer	Number of hidden layers
Wider networks learn more features per layer	Deeper networks learn more complex hierarchical features

Example

- **Wide Network:** Few layers, many neurons.
 - **Deep Network:** Many layers, fewer neurons per layer.
-

4. Dense Layers

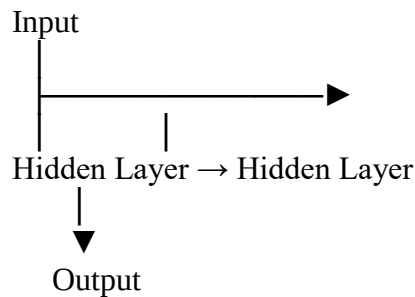
A **Dense Layer** (Fully Connected Layer) connects every neuron in one layer to every neuron in the next layer.

Features

- Most common layer in MLPs.
 - Performs weighted computations.
 - Learns complex relationships.
-

5. Skip Connections (Introduction)

A **Skip Connection** allows information to bypass one or more layers and directly connect to a later layer.



Benefits

- Reduces the vanishing gradient problem.
- Enables training of very deep networks.
- Improves accuracy and faster convergence.

Example: Used in **ResNet (Residual Network)**.

Module 3: Forward Propagation

Introduction

Forward Propagation is the process of passing input data through a neural network to generate the final prediction. During this process, data moves **only in the forward direction** (Input → Hidden Layer(s) → Output Layer).

It is the **first step** in training and prediction.

Input → Hidden Layer(s) → Output

3.1 Forward Pass

Definition

A **Forward Pass** is the process where input data travels through each layer of the neural network to produce an output.

Working Steps

1. Input data is fed into the network.
2. Each neuron calculates the weighted sum.
3. Bias is added.
4. Activation function is applied.
5. Output is passed to the next layer.
6. Final prediction is generated.

Input



Weighted Sum ($Wx + b$)



Activation Function



Hidden Layer



Output Prediction

Formula

Weighted Sum:

$$z = Wx + b$$

Output:

$$a = f(z)$$

Where:

- **W** = Weights
- **x** = Input
- **b** = Bias
- **f()** = Activation Function
- **a** = Output

Purpose

- Generate predictions.
 - Pass information through the network.
 - Calculate output before computing loss.
-

3.2 Matrix Representation

Definition

Instead of calculating each neuron individually, neural networks use **matrices** to perform computations efficiently.

Formula

$$Z=WX+B$$

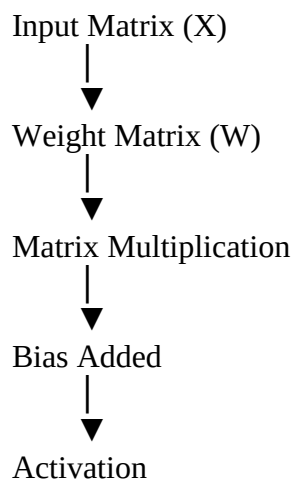
Where:

- **X** = Input Matrix
- **W** = Weight Matrix
- **B** = Bias Matrix
- **Z** = Weighted Sum Matrix

Advantages

- Faster computation.
- Handles multiple neurons simultaneously.
- Optimized for GPU processing.

Example



3.3 Vectorization

Definition

Vectorization means performing calculations on an entire vector or matrix at once instead of processing one value at a time.

Without Vectorization

Input 1 → Calculate

Input 2 → Calculate

Input 3 → Calculate

With Vectorization

All Inputs



Single Matrix Operation

Advantages

- Faster execution.
- Efficient memory usage.
- Makes better use of GPUs.
- Reduces code complexity.

3.4 Batch Processing

Definition

Instead of processing one sample at a time, **Batch Processing** processes multiple samples together.

Example

Dataset = 1000 Images

Batch Size = 100

Batch 1 → 100 Images

Batch 2 → 100 Images

Batch 3 → 100 Images

...

Types

Type	Description
Online Learning	1 sample at a time
Mini-Batch Learning	Small group (most common)
Batch Learning	Entire dataset at once

Advantages

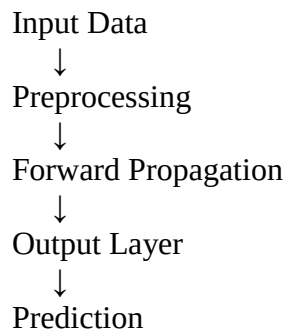
- Faster training.
 - Better GPU utilization.
 - Stable learning.
-

3.5 Prediction Pipeline

Definition

The **Prediction Pipeline** is the complete sequence of steps used to generate predictions from new input data.

Pipeline



Steps

1. Receive input data.
2. Preprocess the data.
3. Pass data through the neural network.
4. Compute the output.
5. Return the final prediction.

Example

Input: Image of a handwritten digit

→ Preprocess image

→ Neural network performs forward propagation

→ **Output:** Predicted digit = 7

Module 4: Activation Functions

Introduction

An **Activation Function** determines whether a neuron should be activated or not. It converts the weighted sum into an output that is passed to the next layer.

Without activation functions, a neural network behaves like a **simple linear model** and cannot learn complex patterns.

Input → Weighted Sum ($Wx+b$) → Activation Function → Output

Why Activation Functions?

- Introduce **non-linearity**.
- Learn complex patterns.
- Improve prediction accuracy.
- Help deep neural networks solve real-world problems.

1. Binary Step Function

Definition

Outputs **0 or 1** based on a threshold.

Formula

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Output Range

0 or 1

Advantages

- Simple
- Fast

Limitations

- Not differentiable
- Cannot be used in backpropagation

Applications

- Early Perceptron models
-

2. Linear Function

Definition

Output is directly proportional to the input.

Formula

$$f(x)=x$$

Output Range

$$(-\infty, +\infty)$$

Advantages

- Simple

Limitations

- No non-linearity
- Deep networks become equivalent to a single layer

Applications

- Regression output layer
-

3. Sigmoid Function

Definition

Maps values between **0 and 1**.

Formula

$$f(x) = 1 / (1 + e^{-x})$$

Output Range

0 to 1

Advantages

- Smooth curve
- Good for probabilities

Limitations

- Vanishing gradient
- Slow training

Applications

- Binary Classification Output Layer
-

4. Tanh (Hyperbolic Tangent)

Definition

Maps values between **-1 and 1**.

Formula

$$f(x) = \tanh(x)$$

Output Range

-1 to 1

Advantages

- Zero-centered output
- Better than Sigmoid

Limitations

- Vanishing gradient

Applications

- Hidden layers (older networks)
-

5. ReLU (Rectified Linear Unit)

Definition

Returns input if positive, otherwise returns 0.

Formula

$$f(x) = \max(0, x)$$

Output Range

0 to $+\infty$

Advantages

- Fast
- Simple
- Most widely used
- Reduces vanishing gradient

Limitations

- Dying ReLU problem

Applications

- Hidden layers in CNNs and MLPs
-

6. Leaky ReLU

Definition

Allows a small negative output instead of zero.

Formula

$$f(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x < 0 \end{cases}$$

Advantages

- Solves Dying ReLU
- Better gradient flow

Applications

- Deep Neural Networks
-

7. PReLU (Parametric ReLU)

Definition

Similar to Leaky ReLU, but the negative slope is **learned automatically** during training.

Advantages

- More flexible
- Better performance

Limitation

- Slightly more parameters
-

8. ELU (Exponential Linear Unit)

Definition

Uses exponential values for negative inputs.

Advantages

- Faster convergence
- Reduces bias shift

Limitation

- More computationally expensive than ReLU
-

9. SELU (Scaled ELU)

Definition

A scaled version of ELU that helps networks **self-normalize**.

Advantages

- Stable training
- Works well in deep networks

Applications

- Self-Normalizing Neural Networks (SNN)
-

10. Softplus

Definition

A smooth version of ReLU.

Formula

$$f(x) = \ln(1 + e^x)$$

Advantages

- Smooth and differentiable
- No Dying ReLU problem

Limitation

- Slower than ReLU
-

11. Softsign

Definition

Produces smooth outputs similar to Tanh.

Formula

$$f(x) = x / (1 + |x|)$$

Output Range

-1 to 1

Advantages

- Smooth
 - Computationally simple
-

12. GELU (Gaussian Error Linear Unit)

Definition

Weights inputs based on probability rather than simply removing negative values.

Advantages

- Better performance than ReLU
- Smooth activation

Applications

- BERT
 - GPT
 - Vision Transformers (ViT)
-

13. Swish

Definition

Proposed by Google.

Formula

$$f(x)=x\times\text{Sigmoid}(x)$$

Advantages

- Smooth
- Better than ReLU in many tasks

Applications

- EfficientNet
-

14. Mish

Definition

A smooth, non-monotonic activation function.

Formula

$$f(x) = x \times \tanh(\ln(1 + e^x))$$

Advantages

- High accuracy
- Smooth gradient
- Better feature learning

Applications

- Advanced Computer Vision models
-

15. Softmax

Definition

Converts multiple outputs into **probability values** whose sum equals **1**.

Output Range

0 to 1

Properties

- All probabilities sum to **1**.
- Highest probability is selected as the prediction.

Advantages

- Ideal for multi-class classification.

Applications

- Image Classification
- Digit Recognition
- Language Models

Activation Function Comparison

Function	Output Range	Main Use	Limitation
Binary Step	0,1	Perceptron	Not differentiable
Linear	$(-\infty, +\infty)$	Regression	No non-linearity
Sigmoid	0–1	Binary Output	Vanishing Gradient
Tanh	-1–1	Hidden Layer	Vanishing Gradient
ReLU	0– ∞	Hidden Layer	Dying ReLU
Leaky ReLU	$(-\infty, \infty)$	Hidden Layer	Small negative slope fixed
PReLU	$(-\infty, \infty)$	Hidden Layer	Extra parameter
ELU	$(-\infty, \infty)$	Hidden Layer	Computationally expensive
SELU	$(-\infty, \infty)$	Deep Networks	Limited use cases
Softplus	0– ∞	ReLU Alternative	Slower
Softsign	-1–1	Hidden Layer	Less common
GELU	$(-\infty, \infty)$	Transformers	More computation
Swish	$(-\infty, \infty)$	Modern Networks	Slower than ReLU
Mish	$(-\infty, \infty)$	Computer Vision	Computationally expensive
Softmax	0–1	Multi-class Output	Output layer only

Which Activation Function Should You Use?

Task	Recommended Activation
Binary Classification	Sigmoid
Multi-Class Classification	Softmax
Hidden Layers (Most Networks)	ReLU
Deep Networks	Leaky ReLU / GELU / Swish
Transformers (BERT, GPT)	GELU
Computer Vision	ReLU / Mish / Swish
Regression Output	Linear

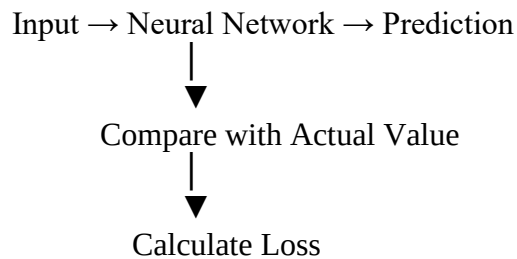
Module 5: Loss Functions

Introduction

A **Loss Function** measures the difference between the **actual output (True Value)** and the **predicted output** of a neural network.

- **Lower Loss** → Better model performance.
- **Higher Loss** → Poor predictions.

The goal of training is to **minimize the loss**.



Types of Loss Functions

1. **Regression Loss Functions** → Used for continuous values.
2. **Classification Loss Functions** → Used for class prediction.

5.1 Regression Loss Functions

Used when the output is a **continuous value** (e.g., price, temperature).

1. Mean Squared Error (MSE)

Definition

Calculates the average of the **squared differences** between actual and predicted values.

Formula

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

Where:

- y = Actual Value
- $\hat{y}$ = Predicted Value
- n = Number of Samples

Advantages

- Simple and widely used.
- Penalizes large errors more.

Limitations

- Sensitive to outliers.

Applications

- House Price Prediction
 - Sales Forecasting
-

2. Mean Absolute Error (MAE)

Definition

Calculates the average of the **absolute differences** between actual and predicted values.

Formula

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

Advantages

- Easy to understand.
- Less sensitive to outliers.

Limitations

- Does not heavily penalize large errors.

Applications

- Weather Prediction
 - Demand Forecasting
-

3. Huber Loss

Definition

Combines the advantages of **MSE** and **MAE**.

- Uses **MSE** for small errors.
- Uses **MAE** for large errors.

Advantages

- Robust to outliers.
- Stable training.

Applications

- Regression with noisy data.
-

4. Log-Cosh Loss

Definition

Uses the logarithm of the hyperbolic cosine of prediction errors.

Advantages

- Smooth and differentiable.
- Less affected by outliers than MSE.

Applications

- Deep Regression Models.
-

Regression Loss Comparison

Loss	Sensitive to Outliers	Best For
MSE	High	General Regression
MAE	Low	Noisy Data
Huber	Medium	Robust Regression
Log-Cosh	Low	Smooth Optimization

5.2 Classification Loss Functions

Used when predicting **categories or classes**.

1. Binary Cross Entropy (BCE)

Definition

Measures error in **binary classification** problems (2 classes).

Output

Probability between **0 and 1**.

Advantages

- Ideal for binary classification.
- Works well with Sigmoid activation.

Applications

- Spam Detection
 - Disease Prediction
 - Fraud Detection
-

2. Categorical Cross Entropy (CCE)

Definition

Used for **multi-class classification** when labels are **one-hot encoded**.

Advantages

- High accuracy for multi-class problems.
- Works with Softmax activation.

Applications

- Image Classification
 - Digit Recognition
-

3. Sparse Categorical Cross Entropy (SCCE)

Definition

Used for multi-class classification when labels are **integers** instead of one-hot encoded.

Example

Label Type	Example
One-Hot	[0,1,0]
Sparse	1

Advantages

- Saves memory.
- Easier label preparation.

Applications

- Large multi-class datasets.
-

4. Hinge Loss

Definition

Measures classification error by maximizing the margin between classes.

Advantages

- Good for margin-based classification.

Applications

- Support Vector Machines (SVM).
-

5. KL Divergence (Kullback-Leibler Divergence)

Definition

Measures how one probability distribution differs from another.

Advantages

- Compares probability distributions.
- Useful in advanced deep learning models.

Applications

- Variational Autoencoders (VAE)
 - Knowledge Distillation
-

6. Focal Loss

Definition

An improved version of Cross Entropy that focuses more on **hard-to-classify samples**.

Advantages

- Handles class imbalance.
- Improves minority class detection.

Applications

- Object Detection
 - Medical Diagnosis
 - Fraud Detection
-

Classification Loss Comparison

Loss	Classes	Common Activation	Applications
Binary Cross Entropy	2	Sigmoid	Binary Classification
Categorical Cross Entropy	Multiple	Softmax	Multi-Class Classification
Sparse Categorical Cross Entropy	Multiple	Softmax	Integer Labels
Hinge Loss	2	Linear	SVM
KL Divergence	Probability Distribution	Softmax	VAE, Distillation
Focal Loss	Binary / Multi-Class	Sigmoid / Softmax	Imbalanced Data

Which Loss Function Should You Use?

Problem	Recommended Loss
House Price Prediction	MSE
Temperature Prediction	MAE
Regression with Outliers	Huber Loss
Binary Classification	Binary Cross Entropy
Multi-Class (One-Hot Labels)	Categorical Cross Entropy
Multi-Class (Integer Labels)	Sparse Categorical Cross Entropy
SVM Classification	Hinge Loss
Probability Distribution Comparison	KL Divergence
Imbalanced Classification	Focal Loss

Module 6: Backpropagation

Introduction

Backpropagation is the learning algorithm used to train neural networks. It calculates how much each **weight** and **bias** contributed to the error and updates them to reduce the loss.

Goal: Minimize the loss by updating weights and biases.

Forward Pass → Calculate Loss → Backpropagation → Update Weights → Repeat

6.1 Why Backpropagation?

Definition

Backpropagation helps the neural network **learn from its mistakes** by updating weights and biases based on the prediction error.

Why Needed?

- Reduces prediction error.
- Improves model accuracy.
- Optimizes weights and biases.
- Enables deep neural networks to learn efficiently.

Example

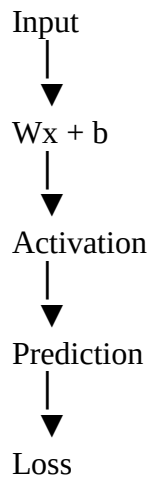
If the actual output is **1** but the model predicts **0.6**, backpropagation adjusts the weights so the next prediction is closer to **1**.

6.2 Computational Graph

Definition

A **Computational Graph** is a visual representation of all mathematical operations performed in a neural network.

It shows how data flows from **input to output**.



Benefits

- Simplifies gradient calculation.
 - Used by TensorFlow and PyTorch for automatic differentiation.
 - Makes backpropagation easier.
-

6.3 Chain Rule

Definition

The **Chain Rule** is a calculus rule used to calculate gradients when multiple functions are connected.

Formula

$$Dy/dx = dy/dz \times dz/dx$$

Purpose

- Computes gradients layer by layer.
- Passes error backward through the network.

Importance

Backpropagation is completely based on the **Chain Rule**.

6.4 Partial Derivatives

Definition

A **Partial Derivative** measures how the loss changes when **one variable** (weight or bias) changes while keeping all other variables constant.

Purpose

- Finds the effect of each weight on the loss.
- Helps determine how weights should be updated.

Notation

$$\partial L / \partial W$$

Where:

- **L** = Loss
 - **W** = Weight
-

6.5 Gradient Computation

Definition

A **Gradient** is the derivative of the loss with respect to a parameter (weight or bias). It indicates the **direction and magnitude** of change needed to reduce the loss.

Interpretation

- **Positive Gradient:** Decrease the parameter.
- **Negative Gradient:** Increase the parameter.
- **Small Gradient:** Close to the minimum.

Purpose

- Guides optimization algorithms.
 - Reduces prediction error.
-

6.6 Weight Update

Definition

Weights are updated after each iteration using the computed gradients.

Formula

$$W_{\text{new}} = W_{\text{old}} - \eta \partial L / \partial W$$

- η (**eta**) = Learning Rate
- L = Loss
- W = Weight

Purpose

- Reduce the loss.
 - Improve future predictions.
-

6.7 Bias Update

Definition

Biases are updated similarly to weights using their gradients.

Formula

$$b_{\text{new}} = b_{\text{old}} - \eta \partial L / \partial b$$

Where:

- b = Bias
- η = Learning Rate

Purpose

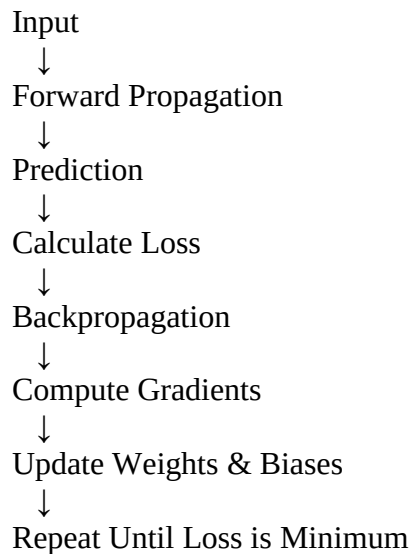
- Fine-tune the neuron's output.

- Improve model accuracy.
-

Advantages

- Faster computation.
 - Efficient GPU utilization.
 - Handles large datasets and deep networks.
-

Backpropagation Process

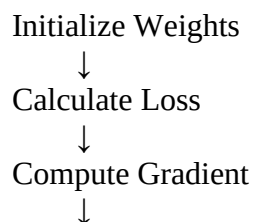


Module 7: Gradient Descent

Introduction

Gradient Descent is an optimization algorithm used to **minimize the loss (cost) function** by updating the model's weights in the direction where the loss decreases.

Goal: Find the optimal weights that produce the minimum loss.



Update Weights



Repeat Until Loss is Minimum

7.1 Concepts

1. Gradient

Definition

A **Gradient** is the derivative (slope) of the loss function. It indicates **how much and in which direction** the weights should change.

- Positive Gradient → Decrease weight.
 - Negative Gradient → Increase weight.
 - Zero Gradient → Minimum point reached.
-

2. Cost Function

Definition

The **Cost Function** measures the overall error of the model.

Purpose

- Evaluates model performance.
- Lower cost means better predictions.

Examples

- MSE
 - Cross Entropy
-

3. Learning Rate (η)

Definition

The **Learning Rate** controls how much the weights are updated during each iteration.

- Small Learning Rate → Slow learning.
- Large Learning Rate → May overshoot the minimum.

Typical values: **0.1, 0.01, 0.001**

4. Epoch

Definition

One complete pass through the **entire training dataset**.

Example:

1000 images processed once = **1 Epoch**

5. Iteration

Definition

One update of model weights after processing a **batch**.

Formula

$$\text{Iterations} = \frac{\text{Training Samples}}{\text{Batch Size}}$$

Example:

1000 Samples

Batch Size = 100

Iterations = 10 per Epoch

6. Batch Size

Definition

The number of training samples processed before updating weights.

Example:

- Batch Size = 32
- Batch Size = 64

- Batch Size = 128

Effect

- Small Batch → More updates, slower but better generalization.
 - Large Batch → Faster training, higher memory usage.
-

7.2 Types of Gradient Descent

1. Batch Gradient Descent

Definition

Uses the **entire dataset** to calculate the gradient before updating weights.

Advantages

- Stable updates.
- Accurate gradient.

Limitations

- Slow for large datasets.
 - High memory usage.
-

2. Stochastic Gradient Descent (SGD)

Definition

Updates weights **after every single training sample**.

Advantages

- Fast updates.
- Escapes local minima more easily.

Limitations

- Noisy and unstable training.
-

3. Mini-Batch Gradient Descent

Definition

Updates weights after processing a **small batch** of data.

Advantages

- Faster than Batch GD.
- More stable than SGD.
- Most commonly used.

Typical Batch Sizes

32, 64, 128, 256

Gradient Descent Comparison

Type	Data Used	Speed	Stability
Batch GD	Entire Dataset	Slow	High
SGD	One Sample	Fast	Low
Mini-Batch GD	Small Batch	Fast	High

7.3 Challenges

1. Local Minima

A point where loss is low compared to nearby points but **not the lowest overall**.

Problem:

- Model may stop before reaching the best solution.
-

2. Global Minima

The **lowest possible value** of the loss function.

Goal:

- Gradient Descent aims to reach the global minimum.

3. Saddle Points

A point where the gradient is close to zero but **is not a minimum**.

Problem:

- Training may slow down or stop.
-

4. Vanishing Gradient

Gradients become **very small**, causing early layers to learn very slowly.

Solution

- ReLU
 - Batch Normalization
 - Residual Networks (ResNet)
-

5. Exploding Gradient

Gradients become **very large**, making training unstable.

Solution

- Gradient Clipping
 - Proper Weight Initialization
 - Batch Normalization
-

Module 8: Optimization Algorithms

Introduction

Optimization algorithms improve Gradient Descent by making training **faster, more stable, and more accurate**.

8.1 SGD (Stochastic Gradient Descent)

Definition

Updates weights after processing **one training sample**.

Advantages

- Fast updates.
- Simple.

Limitation

- Noisy training.
-

8.2 Momentum

Definition

Momentum remembers previous gradients to speed up learning and reduce oscillations.

Advantages

- Faster convergence.
 - Smoother updates.
-

8.3 Nesterov Momentum (NAG)

Definition

An improved version of Momentum that **looks ahead** before updating weights.

Advantages

- Faster convergence.
 - More accurate updates.
-

8.4 AdaGrad (Adaptive Gradient)

Definition

Automatically adjusts the learning rate for each parameter.

Advantages

- Good for sparse data.
- No manual learning rate tuning initially.

Limitation

- Learning rate decreases too much over time.
-

8.5 RMSProp (Root Mean Square Propagation)

Definition

Improves AdaGrad by preventing the learning rate from shrinking too quickly.

Advantages

- Stable learning.
- Faster convergence.

Applications

- CNNs
 - RNNs
-

8.6 AdaDelta

Definition

An extension of RMSProp that adapts learning rates without requiring a manually chosen global learning rate.

Advantages

- No manual learning rate.
 - Stable optimization.
-

8.7 Adam (Adaptive Moment Estimation)

Definition

Combines **Momentum** + **RMSProp**.

Advantages

- Fast convergence.
- Adaptive learning rate.
- Most widely used optimizer.

Applications

- CNN
 - RNN
 - Transformers
 - LLMs
-

8.8 AdamW

Definition

A modified version of Adam with **better weight decay (regularization)**.

Advantages

- Better generalization.
- Reduces overfitting.

Applications

- Vision Transformers
 - BERT
 - GPT
-

8.9 Nadam

Definition

Combines **Adam** + **Nesterov Momentum**.

Advantages

- Faster convergence.
- Better optimization.

Which Optimizer Should You Use?

Problem	Recommended Optimizer
Beginners	Adam
CNN	Adam / RMSProp
RNN / LSTM	RMSProp / Adam
Transformer / LLM	AdamW
Sparse Data	AdaGrad
Large Deep Networks	Momentum / Nesterov
General Deep Learning	Adam

Module 9: Weight Initialization

Introduction

Weight Initialization is the process of assigning initial values to the weights of a neural network before training begins.

Good initialization helps the model:

- Learn faster.
- Avoid vanishing/exploding gradients.
- Achieve better accuracy.

Goal: Start training with appropriate weight values.

9.1 Zero Initialization

Definition

All weights are initialized to **0**.

Example

$W = [0, 0, 0, 0]$

Advantages

- Very simple.

Limitations

- All neurons learn the same thing.
 - Causes **Symmetry Problem**.
 - Not used in practice.
-

9.2 Random Initialization

Definition

Weights are initialized with small random values.

Example

$W = [0.12, -0.08, 0.25]$

Advantages

- Breaks symmetry.
- Allows neurons to learn different features.

Limitations

- Large values $\rightarrow$ Exploding Gradient.
 - Small values $\rightarrow$ Vanishing Gradient.
-

9.3 Xavier Initialization (Glorot Initialization)

Definition

Initializes weights so that the **variance of inputs and outputs remains approximately the same** across layers.

Best For

- Sigmoid
- Tanh

Advantages

- Stable training.
 - Reduces vanishing gradient.
 - Faster convergence.
-

9.4 He Initialization

Definition

Designed specifically for **ReLU-based** activation functions.

Best For

- ReLU
- Leaky ReLU
- PReLU

Advantages

- Prevents vanishing gradients.
 - Faster learning.
 - Most common initialization for ReLU networks.
-

9.5 Lecun Initialization

Definition

Initialization method designed for **SELU** activation.

Best For

- SELU Networks

Advantages

- Maintains stable activations.
 - Supports self-normalizing networks.
-

Topics

1. Symmetry Breaking

Definition

If all neurons start with the **same weights**, they learn the **same features**, making the network ineffective.

Solution

Use **Random**, **Xavier**, or **He Initialization**.

2. Variance Preservation

Definition

Variance Preservation means keeping the output variance nearly constant across layers.

Benefits

- Stable learning.
 - Prevents vanishing and exploding gradients.
 - Faster convergence.
-

Weight Initialization Comparison

Initialization	Best Activation	Advantages
Zero	None	Simple (not recommended)
Random	General	Breaks symmetry
Xavier	Sigmoid, Tanh	Stable variance
He	ReLU, Leaky ReLU	Prevents vanishing gradient
Lecun	SELU	Self-normalizing networks

Module 10: Regularization

Introduction

Regularization is a set of techniques used to **reduce overfitting** and improve a model's ability to generalize to unseen data.

Goal: Improve model performance on new (unseen) data.

10.1 Overfitting

Definition

The model learns the **training data too well**, including noise, resulting in poor performance on new data.

Characteristics

- High training accuracy.
- Low testing accuracy.

Solution

- Dropout
 - L2 Regularization
 - Early Stopping
 - Data Augmentation
-

10.2 Underfitting

Definition

The model is **too simple** to learn the underlying patterns.

Characteristics

- Low training accuracy.
- Low testing accuracy.

Solution

- Increase model complexity.
 - Train for more epochs.
 - Add more features.
-

10.3 Bias-Variance Tradeoff

Bias

Error caused by an overly simple model.

- High Bias $\rightarrow$ Underfitting

Variance

Error caused by excessive sensitivity to training data.

- High Variance $\rightarrow$ Overfitting

Goal

Maintain **Low Bias + Low Variance**.

Regularization Techniques

1. L1 Regularization (Lasso)

Definition

Adds the **absolute value of weights** to the loss function.

Advantages

- Removes unnecessary features.
- Produces sparse models.

Applications

- Feature Selection.
-

2. L2 Regularization (Ridge)

Definition

Adds the **square of weights** to the loss function.

Advantages

- Reduces large weights.

- Prevents overfitting.
 - Most commonly used.
-

3. ElasticNet

Definition

Combines **L1 + L2 Regularization**.

Advantages

- Feature selection.
 - Prevents overfitting.
 - More flexible.
-

4. Weight Decay

Definition

Gradually reduces weight values during training (commonly implemented with L2 regularization).

Advantages

- Controls model complexity.
 - Improves generalization.
-

5. Dropout

Definition

Randomly deactivates a percentage of neurons during training.

Before Dropout:

● ● ● ● ●

After Dropout:

● × ● × ●

Advantages

- Prevents overfitting.
- Improves generalization.

Common Values

- 0.2
 - 0.5
-

6. Early Stopping

Definition

Stops training when the **validation loss stops improving**.

Advantages

- Prevents over-training.
 - Saves computation time.
-

7. Batch Normalization

Definition

Normalizes the output of each batch during training.

Advantages

- Faster training.
 - Stable learning.
 - Allows higher learning rates.
-

8. Layer Normalization

Definition

Normalizes activations **within each sample** instead of across the batch.

Advantages

- Works well with Transformers and RNNs.

- Independent of batch size.
-

9. Data Augmentation

Definition

Artificially increases dataset size by creating modified copies of existing data.

Examples

- Rotation
- Flip
- Crop
- Zoom
- Brightness adjustment

Advantages

- More training data.
 - Reduces overfitting.
-

10. Label Smoothing

Definition

Instead of assigning **100% confidence** to the correct class, assign a slightly smaller value (e.g., 0.9), distributing the remaining probability among other classes.

Advantages

- Prevents overconfidence.
 - Improves generalization.
-

Regularization Techniques Comparison

Technique	Purpose	Best Use
L1	Feature Selection	Sparse Models
L2	Reduce Large Weights	General Deep Learning
ElasticNet	L1 + L2	Mixed Features

Technique	Purpose	Best Use
Weight Decay	Reduce Weight Magnitude	Deep Networks
Dropout	Disable Random Neurons	Prevent Overfitting
Early Stopping	Stop Early	Avoid Overtraining
Batch Normalization	Normalize Batch	CNNs, MLPs
Layer Normalization	Normalize Layer	Transformers, RNNs
Data Augmentation	Increase Dataset	Computer Vision
Label Smoothing	Reduce Overconfidence	Classification

Which Technique Should You Use?

Problem	Recommended Technique
Overfitting	Dropout, L2, Early Stopping
Small Dataset	Data Augmentation
Feature Selection	L1
Deep Networks	Batch Normalization
Transformers	Layer Normalization
Image Classification	Data Augmentation + BatchNorm
Modern Deep Learning	AdamW + Weight Decay + Dropout

Module 11: Hyperparameter Tuning

Introduction

Hyperparameter Tuning is the process of selecting the **best values for hyperparameters** to improve model performance.

Hyperparameters are set **before training**, whereas **parameters (weights & biases)** are learned **during training**.

11.1 Hyperparameters

1. Learning Rate (η)

Definition

Controls how much the weights are updated in each iteration.

- Too Small → Slow learning.
- Too Large → Unstable training.

Common Values: 0.1, 0.01, 0.001

2. Batch Size

Definition

Number of training samples processed before updating weights.

Common Values

- 32
- 64
- 128
- 256

Effect

- Small Batch → Better accuracy, slower training.
 - Large Batch → Faster training, more memory required.
-

3. Epochs

Definition

Number of times the model sees the **entire training dataset**.

Effect

- Few Epochs → Underfitting.
 - Too Many Epochs → Overfitting.
-

4. Hidden Layers

Definition

The number of hidden layers in the neural network.

Effect

- More Layers → Learn complex patterns.
 - Too Many Layers → Higher computation and overfitting risk.
-

5. Hidden Units (Neurons)

Definition

Number of neurons present in each hidden layer.

Effect

- More Neurons → Higher learning capacity.
 - Too Many → Overfitting and slower training.
-

6. Activation Functions

Definition

Determines the output of each neuron.

Common Choices

Activation	Best Use
ReLU	Hidden Layers
Leaky ReLU	Deep Networks
Sigmoid	Binary Output
Softmax	Multi-Class Output
GELU	Transformers

7. Optimizers

Definition

Algorithm used to update model weights.

Popular Choices

- SGD
- RMSProp
- Adam
- AdamW

Adam is the most commonly used optimizer.

8. Weight Initialization

Definition

Method used to assign initial weight values before training.

Common Methods

Initialization	Best For
Xavier	Sigmoid, Tanh
He	ReLU
Lecun	SELU

11.2 Hyperparameter Tuning Methods

1. Grid Search

Definition

Tests **every possible combination** of hyperparameter values.

Advantages

- Finds the best combination.
- Simple to understand.

Limitations

- Very slow.
 - Computationally expensive.
-

2. Random Search

Definition

Randomly selects combinations of hyperparameters instead of testing all.

Advantages

- Faster than Grid Search.
- Good performance with less computation.

Limitation

- May miss the optimal combination.
-

3. Bayesian Optimization

Definition

Uses previous results to intelligently choose the next hyperparameter values.

Advantages

- Fast.
- Efficient.
- Requires fewer trials.

Applications

- Large Deep Learning models.
 - Expensive training tasks.
-

Hyperparameter Tuning Methods Comparison

Method	Speed	Accuracy	Best For
Grid Search	Slow	High	Small Search Space
Random Search	Medium	Good	General Use
Bayesian Optimization	Fast	Very High	Large Models

Module 12: TensorFlow & Keras

Introduction

TensorFlow is an **open-source deep learning framework** developed by **Google** for building and training machine learning models.

Keras is a **high-level API** built on top of TensorFlow that makes model development simple and user-friendly.

12.1 TensorFlow

Definition

TensorFlow provides tools for:

- Building neural networks.
- Training models.
- Deploying AI applications.

Features

- Open-source.
 - GPU/TPU support.
 - Automatic differentiation.
 - Scalable for production.
-

12.2 Tensor

Definition

A **Tensor** is the basic data structure in TensorFlow (similar to NumPy arrays).

Tensor Types

Tensor	Dimension
Scalar	0D
Vector	1D
Matrix	2D
Tensor	3D or Higher

12.3 Variables

Definition

Variables store **trainable parameters** such as weights and biases.

Purpose

- Updated during training.
 - Persistent across iterations.
-

12.4 Operations

Definition

Operations are mathematical computations performed on tensors.

Examples

- Addition
 - Multiplication
 - Matrix Multiplication
 - Activation Functions
-

12.5 Automatic Differentiation

Definition

TensorFlow automatically computes **gradients (derivatives)** required for backpropagation.

Benefits

- No manual derivative calculations.
 - Faster model development.
 - Accurate gradient computation.
-

12.6 Gradient Tape

Definition

GradientTape records operations and automatically computes gradients.

Purpose

- Custom training loops.
 - Gradient calculation.
 - Backpropagation.
-

12.7 Keras

Definition

Keras is TensorFlow's **high-level API** used to build deep learning models with minimal code.

Advantages

- Easy to learn.
 - Fast development.
 - Beginner-friendly.
-

12.8 Sequential API

Definition

Used for models where layers are connected **one after another**.

Input
↓
Dense
↓
Dense
↓
Output

Best For

- Simple feed-forward networks.
-

12.9 Functional API

Definition

Used for building **complex neural networks** with multiple inputs, outputs, or branches.

Advantages

- Flexible architecture.
 - Supports skip connections and shared layers.
-

12.10 Model Subclassing

Definition

Create custom models by inheriting the Model class.

Advantages

- Maximum flexibility.
 - Suitable for advanced architectures.
-

12.11 Layers

Definition

Layers are the building blocks of a neural network.

Common Layers

- Dense
 - Conv2D
 - MaxPooling2D
 - Flatten
 - Dropout
 - LSTM
 - Embedding
-

12.12 Callbacks

Definition

Callbacks perform actions automatically during training.

Common Callbacks

- EarlyStopping
- ModelCheckpoint
- LearningRateScheduler
- TensorBoard

Benefits

- Save best model.
 - Stop overfitting.
 - Monitor training.
-

12.13 Saving Models

Definition

Store the trained model for future use.

Benefits

- Reuse without retraining.
 - Easy deployment.
 - Share trained models.
-

12.14 Loading Models

Definition

Load a previously saved model for prediction or further training.

Uses

- Deployment.
 - Fine-tuning.
 - Inference.
-

12.15 TensorBoard

Definition

TensorBoard is TensorFlow's visualization tool for monitoring model training.

Can Visualize

- Loss
- Accuracy
- Graphs
- Histograms
- Learning Rate
- Images

Benefits

- Monitor training progress.
- Compare experiments.
- Debug models.

TensorFlow vs Keras

TensorFlow	Keras
Deep Learning Framework	High-Level API
More Flexible	Easier to Use
Advanced Features	Beginner Friendly
Custom Training	Quick Model Building

Module 13: Convolutional Neural Networks (CNN)

Introduction

A **Convolutional Neural Network (CNN)** is a deep learning model mainly used for **image processing and computer vision**. CNNs automatically learn features such as **edges, shapes, textures, and objects** from images.

Applications

- Image Classification
 - Object Detection
 - Face Recognition
 - Medical Image Analysis
 - Self-Driving Cars
-

13.1 Image Fundamentals

1. Pixels

Definition

A **Pixel (Picture Element)** is the smallest unit of an image.

- More pixels $\rightarrow$ Higher image quality.
 - Example: 224×224 image = **50,176 pixels**
-

2. RGB Images

Definition

Color images contain **3 channels**:

- **R** $\rightarrow$ Red
- **G** $\rightarrow$ Green
- **B** $\rightarrow$ Blue

Image Shape = Height $\times$ Width $\times$ 3

Example: $224 \times 224 \times 3$

3. Grayscale Images

Definition

Contains only one color intensity channel.

Image Shape = Height $\times$ Width $\times$ 1

Advantages

- Less memory.
 - Faster processing.
-

4. Channels

Definition

Channels represent the color information of an image.

Image Type	Channels
Grayscale	1
RGB	3
RGBA	4

13.2 CNN Building Blocks

1. Convolution

Definition

The convolution operation slides a **kernel (filter)** over the image to extract important features.

Purpose

- Detect edges.
 - Detect textures.
 - Detect shapes.
-

2. Kernel

Definition

A **Kernel** is a small matrix (e.g., 3×3 , 5×5) used during convolution.

Purpose

- Extract image features.
-

3. Filter

Definition

A **Filter** is a kernel whose values are learned during training.

Purpose

Different filters learn different features like:

- Edges
 - Corners
 - Patterns
-

4. Feature Map

Definition

The output produced after applying a filter to the input image.

Purpose

Represents detected features.

5. Padding

Definition

Adding extra pixels around the image before convolution.

Types

- **Valid Padding:** No padding.
- **Same Padding:** Output size remains the same.

Benefits

- Prevents loss of edge information.
 - Controls output size.
-

6. Stride

Definition

Stride is the number of pixels the filter moves each time.

Example

- Stride = 1 → Move 1 pixel.
- Stride = 2 → Move 2 pixels.

Effect

- Larger stride → Smaller output.
 - Smaller stride → More detailed features.
-

7. Receptive Field

Definition

The region of the input image that influences one neuron.

Importance

- Small receptive field → Local features.
 - Large receptive field → Global features.
-

8. Pooling

Definition

Pooling reduces the size of feature maps while preserving important information.

Benefits

- Reduces computation.
 - Reduces overfitting.
 - Makes features more robust.
-

9. Max Pooling

Definition

Selects the **maximum value** from each pooling window.

Advantages

- Keeps strongest features.
 - Most commonly used.
-

10. Average Pooling

Definition

Calculates the **average value** in each pooling window.

Advantages

- Produces smoother feature maps.
-

11. Global Average Pooling (GAP)

Definition

Computes the average of each feature map, producing **one value per channel**.

Advantages

- Fewer parameters.
 - Reduces overfitting.
 - Replaces Flatten in many modern CNNs.
-

12. Flatten Layer

Definition

Converts a **2D feature map** into a **1D vector**.

Purpose

Connects CNN layers to fully connected layers.

13. Fully Connected (Dense) Layer

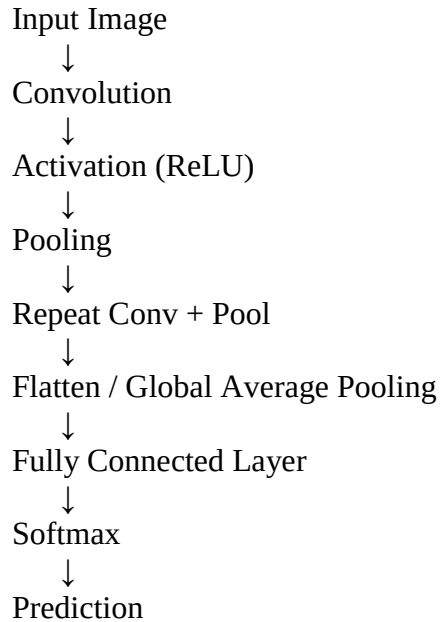
Definition

Uses extracted features to perform the final classification.

Purpose

- Combine learned features.
- Produce final predictions.

Typical CNN Pipeline



13.3 CNN Architectures

Architecture	Key Idea	Main Advantage
LeNet-5	First successful CNN	Handwritten digit recognition
AlexNet	Deep CNN with ReLU & Dropout	Won ImageNet 2012
ZFNet	Improved AlexNet filters	Better feature visualization
VGGNet	Uses repeated 3×3 convolutions	Simple and accurate
GoogLeNet (Inception)	Inception modules	Efficient with fewer parameters
ResNet	Skip (Residual) Connections	Trains very deep networks
ResNeXt	Grouped convolutions	Better accuracy with similar complexity
DenseNet	Dense connections between layers	Strong feature reuse
Xception	Depthwise separable convolution	Efficient and accurate
MobileNet	Lightweight CNN	Mobile & embedded devices
EfficientNet	Balanced scaling of depth, width & resolution	High accuracy with fewer parameters

Module 14: Transfer Learning

Introduction

Transfer Learning is a technique where a model **pretrained on a large dataset** is reused for a new but related task.

Instead of training from scratch, we **transfer the learned knowledge** to solve a new problem.

Benefits

- Faster training.
 - Requires less data.
 - Higher accuracy.
 - Lower computational cost.
-

14.1 Feature Extraction

Definition

Use a pretrained model as a **fixed feature extractor** and train only the final classifier.

Process

Input Image
↓
Pretrained CNN
↓
Extract Features
↓
New Classifier
↓
Prediction

Advantages

- Fast training.
 - Works well with small datasets.
-

14.2 Fine-Tuning

Definition

Fine-tuning means **unfreezing some pretrained layers** and retraining them on the new dataset.

Advantages

- Better accuracy.
- Learns task-specific features.

When to Use

- Medium or large datasets.
 - Dataset is similar to the pretrained model's data.
-

14.3 Frozen Layers

Definition

Frozen layers are layers whose weights **are not updated** during training.

Purpose

- Preserve previously learned features.
 - Reduce training time.
 - Prevent overfitting.
-

14.4 Pretrained Models

Definition

Models already trained on large datasets and ready for reuse.

Popular Pretrained Models

- VGG16
- ResNet50
- InceptionV3
- Xception
- MobileNet
- EfficientNet

Advantages

- High accuracy.
 - Saves training time.
 - Easy to adapt to new tasks.
-

14.5 ImageNet

Definition

ImageNet is one of the world's largest image datasets used to train pretrained CNN models.

Facts

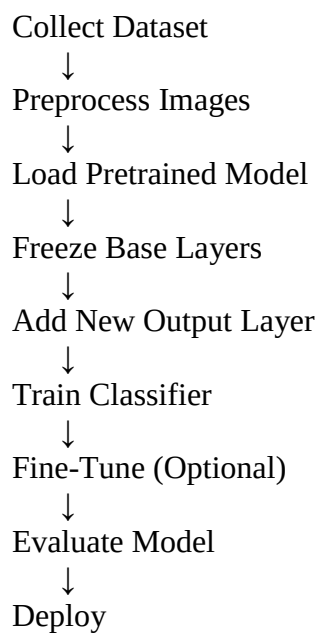
- Millions of images.
- Thousands of object categories.
- Standard benchmark for computer vision.

Importance

Most pretrained CNN models are trained on **ImageNet**.

14.6 Practical Workflow

Step-by-Step Workflow



Transfer Learning Strategies

Strategy	Trainable Layers	Best For
Feature Extraction	Only final layer	Small datasets
Fine-Tuning	Last few or all layers	Larger datasets

Module 15: Object Detection

Introduction

Object Detection is a computer vision task that **identifies objects and locates their positions** in an image using **bounding boxes**.

Unlike image classification (which predicts only the object class), object detection predicts:

- **What** the object is.
- **Where** the object is.

Applications

- Self-driving cars
- Face detection
- Surveillance systems
- Medical imaging
- Traffic monitoring

15.1 Core Concepts

1. Bounding Box

Definition

A **Bounding Box** is a rectangular box drawn around an object to indicate its location.

Components

- x-coordinate
- y-coordinate
- Width
- Height

+-----+

| Object |
+-----+

Purpose

- Locate objects.
- Predict object position.

2. IoU (Intersection over Union)

Definition

IoU measures how much the **predicted bounding box overlaps** with the **actual (ground truth) bounding box**.

Formula

$$\text{IoU} = \frac{\text{Intersection Area}}{\text{Union Area}}$$

Range

- **0** → No overlap.
- **1** → Perfect overlap.

Importance

- Evaluates detection accuracy.
- Higher IoU = Better localization.

3. Non-Maximum Suppression (NMS)

Definition

NMS removes **duplicate bounding boxes** for the same object.

Working

1. Select the box with the highest confidence.
2. Remove overlapping boxes with high IoU.
3. Repeat until only the best boxes remain.

Advantages

- Removes duplicate detections.
 - Improves prediction quality.
-

4. Anchor Boxes

Definition

Anchor Boxes are predefined bounding boxes of different **sizes and aspect ratios** used to detect objects of various shapes.

Advantages

- Detect multiple object sizes.
 - Improve localization accuracy.
-

5. Precision & Recall for Detection

Precision

Measures how many detected objects are **correct**.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall

Measures how many actual objects are **successfully detected**.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Where:

- **TP** = True Positive
- **FP** = False Positive
- **FN** = False Negative

Goal

- High Precision → Few false detections.
 - High Recall → Few missed objects.
-

15.2 Object Detection Models

1. R-CNN (Region-based CNN)

Definition

Generates **region proposals**, then classifies each region using a CNN.

Advantages

- Good accuracy.

Limitations

- Very slow.
 - Separate stages for detection and classification.
-

2. Fast R-CNN

Definition

Processes the entire image **once**, then extracts features for region proposals.

Advantages

- Faster than R-CNN.
- Higher accuracy.

Limitation

- Still depends on external region proposal methods.
-

3. Faster R-CNN

Definition

Introduces a **Region Proposal Network (RPN)** to generate proposals automatically.

Advantages

- Faster.
- End-to-end training.

- High accuracy.
-

4. SSD (Single Shot Detector)

Definition

Detects objects in **one forward pass** using multiple feature maps.

Advantages

- Fast.
 - Good accuracy.
 - Real-time capable.
-

5. YOLO (You Only Look Once)

Definition

YOLO detects objects in **a single pass**, making it one of the fastest object detection algorithms.

Advantages

- Real-time detection.
 - High speed.
 - Good accuracy.
-

YOLO Evolution

Version	Key Improvement
YOLOv1	First real-time detector
YOLOv2	Better accuracy using anchor boxes
YOLOv3	Multi-scale detection
YOLOv4	Faster and more accurate
YOLOv5	Lightweight and easy to use
YOLOv6	Industrial optimization
YOLOv7	Improved speed and accuracy
YOLOv8	Anchor-free, segmentation support

Version	Key Improvement
YOLOv9 / YOLOv10 / YOLOv11	Better efficiency, accuracy, and real-time performance

Object Detection Model Comparison

Model	Speed	Accuracy	Best Use
R-CNN	Slow	High	Research
Fast R-CNN	Medium	High	Detection
Faster R-CNN	Fast	Very High	General Detection
SSD	Very Fast	Good	Real-Time Detection
YOLO	Fastest	High	Real-Time Applications

Module 16: Image Segmentation

Introduction

Image Segmentation divides an image into meaningful regions by assigning a label to **every pixel**.

Unlike object detection (which uses boxes), segmentation provides **pixel-level understanding**.

Applications

- Medical image analysis
 - Autonomous driving
 - Satellite imagery
 - Agriculture
 - Robotics
-

16.1 Types of Image Segmentation

1. Semantic Segmentation

Definition

Assigns the **same class label** to every pixel belonging to the same object category.

Example

If an image contains three cars, all car pixels receive the label **Car**.

Advantage

- Pixel-level classification.

Limitation

- Cannot distinguish different objects of the same class.
-

2. Instance Segmentation

Definition

Separates **each individual object** even if they belong to the same class.

Example

Three cars are labeled as:

- Car 1
- Car 2
- Car 3

Advantage

- Identifies each object separately.
-

3. Panoptic Segmentation

Definition

Combines **Semantic Segmentation** and **Instance Segmentation**.

Output

- Background → Semantic labels.
- Individual objects → Instance labels.

Advantage

- Complete scene understanding.

Segmentation Comparison

Type	Pixel-Level	Distinguishes Instances
Semantic Segmentation	☐	☐
Instance Segmentation	☐	☐
Panoptic Segmentation	☐	☐ (Objects + Background)

16.2 Image Segmentation Models

1. FCN (Fully Convolutional Network)

Definition

The first CNN architecture designed for semantic segmentation.

Advantages

- Pixel-wise prediction.
- Simple architecture.

Limitation

- Low boundary accuracy.
-

2. U-Net

Definition

A U-shaped CNN with **encoder** and **decoder** connected by skip connections.

Advantages

- Excellent localization.
- Works well with small datasets.

Applications

- Medical image segmentation.
- Biomedical imaging.

3. Mask R-CNN

Definition

Extends **Faster R-CNN** by adding a branch to predict **segmentation masks**.

Advantages

- Object detection + Instance segmentation.
 - High accuracy.
-

4. DeepLab

Definition

A semantic segmentation model that uses **Atrous (Dilated) Convolutions** to capture larger context without reducing resolution.

Advantages

- High segmentation accuracy.
- Handles multi-scale objects effectively.

Applications

- Autonomous driving.
 - Satellite image analysis.
-

Segmentation Models Comparison

Model	Segmentation Type	Main Advantage
FCN	Semantic	First end-to-end segmentation model
U-Net	Semantic	Best for medical images
Mask R-CNN	Instance	Detection + Segmentation
DeepLab	Semantic	High accuracy with atrous convolution

Module 17: Recurrent Neural Networks (RNN)

Introduction

A **Recurrent Neural Network (RNN)** is a type of neural network designed to process **sequential data**, where the order of data matters.

Unlike traditional neural networks, RNNs have **memory**, allowing them to use information from previous time steps.

Applications

- Language Translation
 - Speech Recognition
 - Sentiment Analysis
 - Time Series Forecasting
 - Text Generation
-

17.1 Sequential Data

Definition

Sequential data is data where the **order of elements is important**.

Examples

- Sentences
- Audio signals
- Stock prices
- Weather data
- Video frames

Characteristics

- Data is processed one step at a time.
 - Previous information influences future predictions.
-

17.2 Hidden State

Definition

The **Hidden State** stores information from previous time steps and acts as the memory of the RNN.

Purpose

- Remember previous inputs.
- Capture sequence patterns.

Input $\rightarrow$ Hidden State $\rightarrow$ Output
 $\uparrow$
 Previous Hidden State

17.3 Time Steps

Definition

A **Time Step** represents one position in a sequence.

Example

Sentence: "**I Love AI**"

Time Step	Input
t ₁	I
t ₂	Love
t ₃	AI

17.4 Unrolling

Definition

Unrolling represents an RNN across multiple time steps, showing how information flows through the sequence.

$x_1 \rightarrow h_1 \rightarrow y_1$
 $\downarrow$
 $x_2 \rightarrow h_2 \rightarrow y_2$
 $\downarrow$
 $x_3 \rightarrow h_3 \rightarrow y_3$

Purpose

- Visualize sequence processing.

- Perform Backpropagation Through Time (BPTT).
-

17.5 Backpropagation Through Time (BPTT)

Definition

BPTT is the training algorithm for RNNs. It extends backpropagation by computing gradients **across all time steps**.

Purpose

- Update weights.
 - Learn sequence patterns.
 - Minimize loss.
-

17.6 Vanishing Gradient

Definition

Gradients become **very small** while moving backward through many time steps.

Effects

- Early information is forgotten.
- Long-term dependencies are difficult to learn.

Solutions

- LSTM
 - GRU
 - Gradient Clipping
 - Proper Initialization
-

17.7 Exploding Gradient

Definition

Gradients become **very large**, causing unstable weight updates.

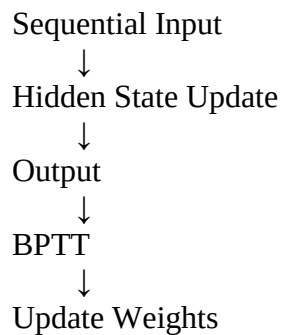
Effects

- Training becomes unstable.
- Loss may increase dramatically.

Solutions

- Gradient Clipping
 - Proper Weight Initialization
 - Lower Learning Rate
-

RNN Workflow



Module 18: Long Short-Term Memory (LSTM)

Introduction

Long Short-Term Memory (LSTM) is a special type of RNN designed to **learn long-term dependencies** by overcoming the **vanishing gradient problem**.

LSTM achieves this using a **Cell State** and three **gates** that control the flow of information.

Applications

- Machine Translation
 - Speech Recognition
 - Text Generation
 - Time Series Prediction
 - Chatbots
-

18.1 Cell State

Definition

The **Cell State** is the long-term memory of an LSTM. It carries important information through the sequence.

Purpose

- Preserve useful information.
 - Forget unnecessary information.
-

18.2 Forget Gate

Definition

The **Forget Gate** decides **what information should be removed** from the Cell State.

Purpose

- Remove irrelevant information.
 - Retain important knowledge.
-

18.3 Input Gate

Definition

The **Input Gate** decides **what new information should be stored** in the Cell State.

Purpose

- Add useful information.
 - Update memory.
-

18.4 Output Gate

Definition

The **Output Gate** decides **what information should be passed to the next hidden state and output**.

Purpose

- Generate the current output.

- Pass relevant information to the next time step.

18.5 Information Flow

The three gates work together to control information flow.

Previous Cell State



Forget Gate



Input Gate



Updated Cell State



Output Gate



Hidden State & Output

18.6 Mathematical Formulation

Forget Gate

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input Gate

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Candidate Cell State

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

Cell State Update

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Hidden State

$$h_t = o_t \odot \tanh(C_t)$$

Where:

- σ = Sigmoid activation.
- $\tanh$ = Hyperbolic tangent.
- $\odot$ = Element-wise multiplication.

18.7 Advantages over RNN

RNN	LSTM
Short-term memory	Long-term memory
Suffers from vanishing gradient	Reduces vanishing gradient
Simple architecture	Uses Cell State & Gates
Poor for long sequences	Excellent for long sequences

Advantages of LSTM

- Learns long-term dependencies.
- Handles long sequences effectively.
- Reduces vanishing gradient.
- Produces better accuracy for sequential tasks.

RNN vs LSTM

Feature	RNN	LSTM
Memory	Short	Long
Long-Term Learning	Poor	Excellent
Vanishing Gradient	Yes	Mostly Solved
Gates	No	Forget, Input, Output
Performance	Lower	Higher

Module 19: Gated Recurrent Unit (GRU)

Introduction

A **Gated Recurrent Unit (GRU)** is an improved version of RNN that solves the **vanishing gradient problem**. It is similar to **LSTM** but uses **fewer gates**, making it faster and simpler.

Applications

- Machine Translation
- Text Classification

- Speech Recognition
 - Time Series Forecasting
-

19.1 Update Gate

Definition

The **Update Gate** decides **how much previous information should be kept** and how much new information should be added.

Purpose

- Retains useful past information.
 - Updates memory when needed.
-

19.2 Reset Gate

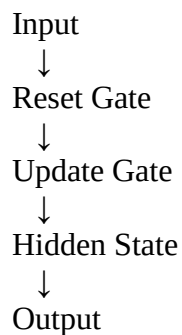
Definition

The **Reset Gate** decides **how much past information should be forgotten**.

Purpose

- Removes unnecessary information.
 - Focuses on the current input.
-

19.3 Information Flow



Working

1. Reset Gate filters old information.
2. Update Gate combines old and new information.
3. Hidden State is updated.
4. Output is generated.

19.4 GRU vs LSTM

Feature	GRU	LSTM
Gates	2	3
Cell State	☐	☐
Hidden State	☐	☐
Speed	Faster	Slower
Memory Usage	Lower	Higher
Long Sequences	Good	Better

19.5 Advantages and Limitations

Advantages

- Faster training.
- Simpler architecture.
- Fewer parameters.
- Reduces vanishing gradient.

Limitations

- Less flexible than LSTM.
- May perform slightly worse on very long sequences.

Module 20: Sequence-to-Sequence (Seq2Seq) Models

Introduction

A **Sequence-to-Sequence (Seq2Seq)** model converts one sequence into another sequence.

Applications

- Machine Translation
- Text Summarization

- Chatbots
 - Speech Recognition
-

20.1 Encoder

Definition

The **Encoder** reads the input sequence and converts it into a fixed-length representation called the **Context Vector**.

Purpose

- Understand input sequence.
 - Extract important information.
-

20.2 Decoder

Definition

The **Decoder** uses the Context Vector to generate the output sequence one token at a time.

Purpose

- Produce translated/generated sequence.
-

20.3 Context Vector

Definition

The **Context Vector** is a compressed representation of the entire input sequence.

Limitation

For long sequences, it may lose important information.

20.4 Teacher Forcing

Definition

During training, the decoder receives the **correct previous output** instead of its own prediction.

Advantages

- Faster learning.
- Better convergence.

Limitation

Training and inference behave differently.

20.5 Inference

Definition

During inference (testing), the decoder uses **its own previous prediction** to generate the next output.

Purpose

- Generate sequences without true labels.
-

20.6 Beam Search (Basic Concept)

Definition

Beam Search keeps the **top K most likely sequences** instead of selecting only the best one at each step.

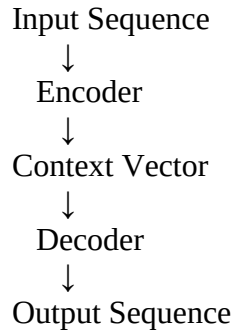
Advantages

- Better sequence generation.
- Higher prediction quality.

Limitation

- More computation than greedy search.
-

Seq2Seq Workflow



Module 21: Attention Mechanism

Introduction

The **Attention Mechanism** allows the model to **focus on the most important parts of the input sequence** while generating each output.

It was introduced to overcome the limitations of a single Context Vector in Seq2Seq models.

21.1 Why Attention?

Problem

A single Context Vector cannot store all information for long sequences.

Solution

Attention lets the decoder access **all encoder outputs**, not just one vector.

Benefits

- Better translation.
 - Handles long sequences.
 - Improves accuracy.
-

21.2 Context Vector Problem

Definition

Compressing a long sequence into one fixed vector causes **information loss**.

Solution

Attention computes a **new context vector for each output step**.

21.3 Attention Scores

Definition

Attention Scores measure **how important each input token is** for generating the current output.

Purpose

- Assign importance to each input.
 - Focus on relevant words.
-

21.4 Alignment

Definition

Alignment determines **which input tokens correspond to each output token**.

Example

English: *I love AI*

French: *J'aime l'IA*

The model aligns **"love"** ↔ **"aime"**.

21.5 Bahdanau Attention

Definition

Also called **Additive Attention**.

Features

- Calculates attention before prediction.
 - Works well with long sequences.
-

21.6 Luong Attention

Definition

Also called **Multiplicative (Dot-Product) Attention**.

Features

- Faster computation.
 - Simpler than Bahdanau Attention.
-

21.7 Self-Attention (Concept Only)

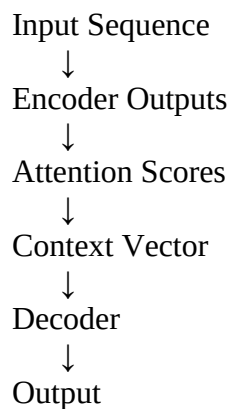
Definition

Self-Attention allows each word in a sequence to **focus on other words in the same sequence**.

Advantages

- Captures long-range relationships.
 - Learns context efficiently.
 - Foundation of Transformer models.
-

Attention Workflow



Module 22: Autoencoders

Introduction

An **Autoencoder** is an unsupervised neural network that learns to **compress data and reconstruct it**.

Applications

- Dimensionality Reduction
 - Image Denoising
 - Anomaly Detection
 - Data Compression
-

22.1 Encoder

Definition

The **Encoder** compresses the input into a smaller representation called the **Latent Space**.

Purpose

- Learn important features.
 - Reduce dimensionality.
-

22.2 Decoder

Definition

The **Decoder** reconstructs the original input from the Latent Space.

Purpose

- Rebuild the input.
 - Minimize reconstruction error.
-

22.3 Latent Space

Definition

The **Latent Space** is the compressed representation of the input.

Purpose

- Store essential features.
 - Remove redundant information.
-

22.4 Reconstruction Loss

Definition

Measures the difference between the **original input** and the **reconstructed output**.

Common Losses

- Mean Squared Error (MSE)
- Binary Cross Entropy (BCE)

Goal

Minimize reconstruction loss.

22.5 Types of Autoencoders

1. Vanilla Autoencoder

Definition

Basic autoencoder with one encoder and one decoder.

Applications

- Data compression.
 - Feature learning.
-

2. Sparse Autoencoder

Definition

Encourages only a few neurons to be active.

Advantages

- Learns meaningful features.

- Reduces redundancy.
-

3. Denoising Autoencoder

Definition

Receives a **noisy input** and learns to reconstruct the **clean original**.

Applications

- Image denoising.
 - Noise removal.
-

4. Contractive Autoencoder

Definition

Adds a regularization term to make the learned representation robust to small input changes.

Advantages

- Stable feature learning.
 - Improved robustness.
-

5. Variational Autoencoder (VAE) – Basic Introduction

Definition

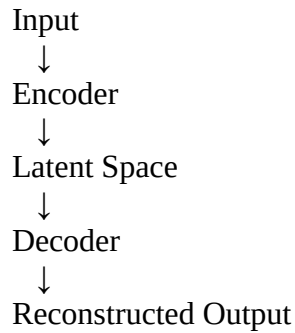
A **Variational Autoencoder (VAE)** learns a **probability distribution** in the latent space instead of a fixed representation.

Advantages

- Learns smooth latent representations.
- Can generate new samples similar to the training data.

Note: VAE is an introduction only here. Advanced generative modeling is covered separately in GenAI topics.

Autoencoder Workflow



Types of Autoencoders Comparison

Type	Main Purpose	Applications
Vanilla	Basic reconstruction	Compression
Sparse	Learn sparse features	Feature Extraction
Denoising	Remove noise	Image Restoration
Contractive	Robust features	Representation Learning
Variational (VAE)	Learn probability distribution	Basic Generative Modeling

Module 23: Graph Neural Networks (GNN)

Introduction

A **Graph Neural Network (GNN)** is a deep learning model designed to work with **graph-structured data**, where data is represented as **nodes** and **edges**.

Unlike CNNs (images) and RNNs (sequences), GNNs learn **relationships between connected entities**.

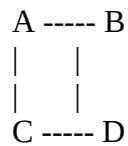
Applications

- Social Networks
 - Recommendation Systems
 - Fraud Detection
 - Molecule & Drug Discovery
 - Knowledge Graphs
-

23.1 Graph Representation

Definition

A **Graph** is a data structure consisting of **Nodes (Vertices)** and **Edges (Connections)**.



Components

- Nodes → Objects
 - Edges → Relationships
-

23.2 Nodes

Definition

Nodes represent **entities or objects** in a graph.

Examples

- Person (Social Network)
 - User (Recommendation System)
 - City (Road Network)
 - Atom (Molecule)
-

23.3 Edges

Definition

Edges represent the **relationship or connection** between nodes.

Examples

- Friendship
 - Road Connection
 - Communication
 - Chemical Bond
-

23.4 Adjacency Matrix

Definition

An **Adjacency Matrix** is a matrix that represents graph connections.

Example

	A	B	C
A	0	1	1
B	1	0	0
C	1	0	0

- **1** → Connected
 - **0** → Not Connected
-

23.5 Message Passing

Definition

Message Passing is the process where each node **collects information from its neighboring nodes** and updates its representation.

Steps

1. Gather neighbor information.
2. Aggregate messages.
3. Update node representation.

Purpose

- Learn graph structure.
 - Capture relationships.
-

23.6 Graph Convolutional Network (GCN)

Definition

A **GCN** applies convolution operations on graph data by aggregating information from neighboring nodes.

Advantages

- Simple architecture.
 - Effective for node classification.
 - Widely used.
-

23.7 Graph Attention Network (GAT)

Definition

A **GAT** uses an **attention mechanism** to assign different importance (weights) to neighboring nodes.

Advantages

- Learns important neighbors.
 - Better performance on complex graphs.
-

23.8 GraphSAGE

Definition

GraphSAGE learns node representations by **sampling and aggregating neighboring nodes**.

Advantages

- Scalable to large graphs.
 - Supports unseen nodes.
-

Popular GNN Models Comparison

Model	Main Idea	Advantage
GCN	Graph Convolution	Simple and effective
GAT	Attention on neighbors	Learns neighbor importance
GraphSAGE	Neighbor sampling	Scalable to large graphs

Module 24: Model Evaluation

Introduction

Model Evaluation measures how well a deep learning model performs on unseen data.

The choice of evaluation metric depends on the **type of problem**:

- **Classification** → **Classification Metrics**
 - **Regression** → **Regression Metrics**
-

24.1 Classification Metrics

1. Accuracy

Definition

Accuracy is the percentage of **correct predictions**.

Formula

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Best For

Balanced datasets.

Limitation

Can be misleading for imbalanced datasets.

2. Precision

Definition

Precision measures how many predicted positive samples are actually positive.

Formula

$$\text{Precision} = \frac{TP}{TP + FP}$$

Importance

- Reduces false positives.
- Useful when false alarms are costly.

3. Recall

Definition

Recall measures how many actual positive samples are correctly detected.

Formula

$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

Importance

- Reduces false negatives.
 - Important in disease detection.
-

4. F1-Score

Definition

F1-Score is the **harmonic mean** of Precision and Recall.

Formula

$\text{F1} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

Importance

Useful for **imbalanced datasets**.

5. Confusion Matrix

Definition

A table showing actual vs predicted classes.

Actual / Predicted	Positive	Negative
Positive	TP	FN
Negative	FP	TN

Where:

- **TP** → True Positive
- **TN** → True Negative
- **FP** → False Positive
- **FN** → False Negative

Purpose

- Analyze model errors.
 - Compute evaluation metrics.
-

6. ROC Curve

Definition

The **Receiver Operating Characteristic (ROC)** Curve plots:

- **True Positive Rate (Recall)** vs.
- **False Positive Rate (FPR)**

Purpose

Evaluate classification performance at different thresholds.

7. AUC (Area Under Curve)

Definition

AUC measures the area under the ROC Curve.

Range

- **1.0** → Perfect classifier.
- **0.5** → Random guessing.

Higher AUC = Better Model

24.2 Regression Metrics

1. Mean Absolute Error (MAE)

Definition

Average absolute difference between predicted and actual values.

Advantages

- Easy to interpret.
 - Less sensitive to outliers.
-

2. Mean Squared Error (MSE)

Definition

Average of squared prediction errors.

Advantages

- Penalizes large errors.
 - Common training loss.
-

3. Root Mean Squared Error (RMSE)

Definition

Square root of MSE.

Advantages

- Same unit as target variable.
 - Easier to interpret than MSE.
-

4. R² Score (Coefficient of Determination)

Definition

Measures how much variance in the target variable is explained by the model.

Range

- **1** → Perfect prediction.
- **0** → No improvement over mean prediction.
- **< 0** → Worse than predicting the mean.

Higher R^2 = Better Model

24.3 Deep Learning Evaluation

1. Learning Curves

Definition

Graphs showing **training** and **validation** performance over epochs.

Purpose

- Detect overfitting.
 - Detect underfitting.
 - Monitor training progress.
-

2. Validation Curves

Definition

Show how model performance changes with different **hyperparameter values**.

Purpose

- Select the best hyperparameters.
 - Improve model performance.
-

3. Error Analysis

Definition

The process of analyzing **incorrect predictions** to improve the model.

Steps

1. Identify wrong predictions.

2. Find the cause of errors.
3. Improve data or model.
4. Retrain and evaluate.

Benefits

- Improves accuracy.
 - Helps identify data quality issues.
-

Classification Metrics Comparison

Metric	Best Use	Higher/Lower
Accuracy	Balanced data	Higher
Precision	Reduce False Positives	Higher
Recall	Reduce False Negatives	Higher
F1-Score	Imbalanced data	Higher
AUC	Overall classifier quality	Higher

Regression Metrics Comparison

Metric	Meaning	Better Value
MAE	Average absolute error	Lower
MSE	Average squared error	Lower
RMSE	Square root of MSE	Lower
R ² Score	Explained variance	Higher